

Error Propagation in Multi-Step Agentic AI Workflows for Research Automation

Abstract

Autonomous agentic AI workflows—built on Large Language Models (LLMs) configured for multi-step reasoning, tool execution, and dynamic planning—are rapidly transforming scientific research automation. However, the transition from single-prompt generation to complex, multi-agent research pipelines introduces a critical systemic vulnerability: agentic error propagation. In multi-step research topologies, minor downstream inaccuracies, tool invocation failures, or context hallucinations do not remain localized; rather, they accumulate, self-condition, and amplify across intermediate stages, leading to catastrophic system-level failures. This paper presents a comprehensive theoretical and empirical investigation into error propagation dynamics within automated scientific discovery workflows. We establish mathematical models of cascading failure across Directed Acyclic Graph (DAG) execution traces, analyze the self-conditioning mechanism wherein LLMs amplify prior errors present in their working memory, and evaluate the efficacy of current mitigation paradigms including cross-verification, Standardized Operating Procedures (SOPs), and progressive agentic tree search. Finally, we outline key research gaps and propose future directions toward robust, self-correcting agent architectures for scientific discovery.

Keywords: Agentic AI, Autonomous Scientific Discovery, Error Propagation, Cascading Failures, Multi-Agent Systems, Large Language Models, Reliability Engineering.

1. Introduction

The paradigm of automated scientific discovery has advanced dramatically from rule-based symbolic engines to autonomous, multi-step agentic AI systems. Contemporary agentic frameworks—such as *The AI Scientist*, *ChemCrow*, and *MetaGPT*—combine Large Language Models (LLMs) with tool execution, code generation, web retrieval, and multi-agent interaction loops. These workflows enable AI systems to formulate hypotheses, design and execute code-based experiments, analyze results, and synthesize scientific manuscripts with minimal human oversight.

Despite these capabilities, production deployments of complex agentic workflows exhibit high failure rates, often ranging between 70% and 95% on multi-step end-to-end tasks. The primary driver of this failure regime is not necessarily a lack of baseline domain knowledge, but rather the mathematical reality of **error propagation and cascading failure**. Unlike single-turn prompt-response interactions where errors are isolated to a single inference pass, multi-step agentic research workflows operate sequentially or graph-computationally. The output of step k serves as the input, context, or tool parameter for step $k+1$.

such as *AgentBench* (Liu et al., 2023) and *WebArena* (Zhou et al., 2023) demonstrated that state-of-the-art LLM agents drop sharply in success rate when tasks require long-horizon planning exceeding 5 to 10 sequential actions.

In multi-step logic contexts, ICML research on cascading failure dynamics identified that once an intermediate error occurs, the probability of subsequent steps containing new errors escalates drastically. This compounding degradation—often termed "compound interest in reverse"—represents the baseline bottleneck limiting full autonomy in scientific discovery pipelines.

3. Theoretical Framework and Mathematical Formalization

To rigorously analyze how errors compound across autonomous scientific discovery pipelines, we model an agentic workflow as a Directed Acyclic Graph (DAG) $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where nodes $v_i \in \mathcal{V}$ represent discrete execution steps (e.g., hypothesis generation, tool calls, code compilation, data aggregation) and directed edges $(v_i, v_j) \in \mathcal{E}$ represent dependencies and context flows.

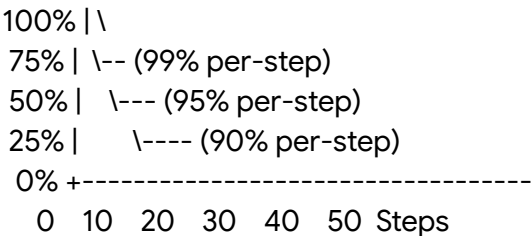
3.1 The Classical Multiplicative Decay Model

Consider a sequential linear workflow of N steps ($v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_N$). Assuming conditionally independent step successes where step i has an individual accuracy $p_i = 1 - \epsilon_i$ (with error rate $\epsilon_i \in [0, 1]$), the overall workflow success rate S_N is given by:

$$S_N = \prod_{i=1}^N (1 - \epsilon_i)$$

If each step maintains a constant success probability $p = 0.95$, the success rate of a 10-step research pipeline is $S_{10} = (0.95)^{10} \approx 0.5987$ (59.9%). In a 50-step end-to-end scientific workflow (e.g., multi-trial hyperparameter search and data synthesis), $S_{50} = (0.95)^{50} \approx 0.0769$ (7.7%).

Workflow Success Rate vs. Number of Sequential Steps



3.2 The Self-Conditioning Markovian Error Cascade

The independent assumption, however, underestimates real-world agent degradation. In autoregressive language models, agents condition their next generation on the full context history in working memory. When step k contains an uncorrected hallucination or logical flaw, step $k+1$ generates tokens conditioned on corrupted prompt context.

We model this as a non-homogeneous Markov chain with conditional transition probabilities $P(E_k \mid E_{k-1})$, where E_k is the indicator event that step k incurs an error.

Empirically, the conditional error rate given a prior error is strictly greater than the baseline error rate:

$$P(E_k \mid E_{k-1}) \gg P(E_k \mid \neg E_{k-1})$$

To quantify this, we define the **Amplification Factor (AF)** for step k as the risk ratio:

$$AF_k = \frac{P(E_k \mid E_{k-1})}{P(E_k \mid \neg E_{k-1})}$$

When $AF_k > 1$, the workflow exhibits positive error feedback. Recent trace-level evaluations reveal that empirical AF values across frontier language models range between 3.29 and 7.79 on multi-step reasoning steps, indicating that prior errors make subsequent step failures roughly 3 to 8 times more likely.

3.3 Bayesian Decomposition of Automation Risk

Building on risk framework formalizations (Srivastava & Sah), the total expected loss $\mathbb{E}[\text{Loss}]$ of an autonomous research task subject to execution error F , hallucination state H , and scientific severity S conditioned on domain context A is modeled as:

$$\mathbb{E}[\text{Loss}] = P(F) \times P(H \mid F, A) \times \mathbb{E}[S \mid H]$$

Where:

- $P(F)$ is the probability of an unhandled step-level execution or tool failure.
- $P(H \mid F, A)$ is the probability that execution failure F induces context hallucination H given agent autonomy A .
- $\mathbb{E}[S \mid H]$ is the expected scientific severity of the failure (e.g., publishing invalid experimental claims or executing unsafe chemical actions).

4. Main Thematic and Technical Discussion: Error Modes in Scientific Workflows

Error propagation in scientific research automation exhibits unique structural patterns distinct from standard conversational LLMs. We identify four primary error propagation pathways in multi-step research topologies:

AGENTIC ERROR PROPAGATION PATHWAYS		
Pathway	Propagation Mechanism	
1. Literature & Context	Hallucinated paper claims --> Invalid	
Contamination	theoretical premise --> Flawed baseline	
2. Code & Tool Execution	Unhandled API/runtime exception --> Silent	
State Corruption	output mutation --> Corrupted plot/metrics	
3. Multi-Agent Debate	Agent A hallucination --> Agent B confirmation	
False Consensus	--> Unanimous approval of invalid findings	
4. Long-Horizon Window	Truncation/context drop --> State drift	
Tracking Failure	--> Repeated/skipped experimental stages	

4.1 Literature Retrieval and Context Contamination

In automated survey generation or hypothesis formulation, agents query scientific search engines (e.g., Semantic Scholar, arXiv APIs). Context contamination occurs when:

1. The retrieval agent parses irrelevant or hallucinated citation metadata.
2. The hypothesis formulation agent incorporates this baseline as factual ground truth.
3. The downstream experiment planning agent constructs empirical hypotheses around non-existent scientific phenomena.

Because downstream agents possess no intrinsic ground-truth verification engine outside their provided context, the error becomes structurally unrecoverable within the prompt history.

4.2 Code Execution Sandbox & Environmental State Drift

In platforms like *The AI Scientist*, agents modify code repositories, execute bash scripts, and train neural models. When a code modification fails silently (e.g., catching an exception without terminating, or failing to converge due to an unseen variable initialization bug), the agent receives non-null execution outputs.

The system state drifts: the agent assumes the code executed correctly and feeds bad metrics (e.g., accuracy = 0.0 or random loss values) into its scientific drafting module. The drafting agent then synthesizes an elaborate theoretical argument explaining why the experimental technique failed or succeeded, converting a simple software runtime error into a persistent conceptual hallucination.

4.3 Multi-Agent Debate and False Consensus Cascades

Multi-agent research systems often utilize peer-review or debate structures (e.g., Writer Agent, Reviewer Agent, Meta-Reviewer Agent) to filter errors. However, without external grounding (such as deterministic code execution or hardware verification), multi-agent debate can amplify errors through **false consensus**.

If Agent A provides a plausible but flawed derivation, Agent B (acting as a reviewer) conditions its analysis on Agent A's output. If Agent B agrees with the plausible phrasing, the error becomes entrenched in the collective system memory. This dynamic creates a positive feedback loop of mutual reinforcement, increasing overall confidence in erroneous conclusions.

4.4 Position Vulnerability and Context Window Overrun

In multi-step execution traces, step index position strongly correlates with failure impact. Early-stage errors (e.g., hypothesis design) contaminate the entire downstream graph execution. Conversely, late-stage errors (e.g., manuscript formatting) have lower topological impact but are harder to correct because working memory buffers are saturated with past reasoning tokens. As context windows fill, agents suffer from context drift, dropping early constraints (e.g., baseline parameters or safety thresholds) specified at step 0.

5. Current Approaches and Mitigation Developments

To counteract cascading failure amplification, current literature has evolved several architectural, algorithmic, and governance interventions.

5.1 Standardized Operating Procedures (SOPs) and Structured Handoffs

As introduced in *MetaGPT*, restricting free-form natural language communication between agents significantly mitigates error transmission. By enforcing strict schema constraints (e.g., JSON schemas, structured markdown PRDs, formal API contracts), systems prevent conversational ambiguity and eliminate "telephone game" distortions. Intermediate validation layers enforce type checking and structural completeness before passing handoffs downstream.

5.2 Verification Interventions: Cross-Verification vs. Step-Level Self-Checking

A key discovery in recent multi-step reasoning evaluations is the stark divergence in efficacy between self-reflection and cross-verification:

- **Step-Level Self-Checking (Harmful/Ineffective):** Requesting an agent to check its own immediate output ($E_k \rightarrow \text{SelfCheck}(E_k)$) often *degrades* performance. Because the model conditions on its own prior reasoning context, it frequently rationalizes its initial error, increasing confirmation bias and reducing baseline accuracy (e.g., dropping benchmark accuracy from 52% to 36%).
- **Independent Cross-Verification (Effective):** Delegating verification to a distinct model

instance, a separate specialized verifier agent, or a deterministic tool (e.g., formal code execution, unit testing, external mathematical solver) dramatically reduces cascade rates (e.g., lowering failure rates from 24% to 9%).

VERIFICATION ARCHITECTURES	
A) STEP-LEVEL SELF-CHECKING (High failure / confirmation bias risk)	
[Agent Step k] ---> [Self-Evaluate] ---> (Reinforces original error)	
B) INDEPENDENT CROSS-VERIFICATION (Effective cascade mitigation)	
[Agent Step k] ---> [Isolated Verifier / Deterministic Tool] ---> [Pass/Fail]	

5.3 Progressive Agentic Tree Search and Rollback Systems

Advanced discovery platforms, such as *The AI Scientist-v2*, replace linear execution chains with tree-search topologies (e.g., Progressive Agentic Tree Search) managed by dedicated manager agents.

When an experimental node yields runtime exceptions, performance degradation, or invalid outputs, the manager agent does not append the failure to the linear context. Instead, it prunes the failed branch, rolls back the software state to the parent node in the execution tree, and instantiates an alternative hypothesis path. This prevents corrupted execution traces from contaminating the primary prompt history.

6. Research Challenges and Limitations

Despite progress in agent design, fundamental technical limitations hamper complete error mitigation in autonomous research workflows:

- Verification Asymmetry in Scientific Discovery:** In standard code generation, unit tests provide objective, deterministic ground truth. In novel scientific discovery, ground truth is intrinsically unknown. Verifying whether a newly proposed scientific hypothesis or theoretical synthesis is correct requires physical laboratory execution or deep mathematical proof, making AI-based peer review susceptible to ungrounded agreement.
- Context Window Saturated Self-Conditioning:** Even with advanced context compression, long-horizon agents inevitably retain prior reasoning steps in their active window. Once an undetected error enters working memory, transformer self-attention mechanisms allocate weight to that error token sequence, exponentially inflating the likelihood of downstream hallucinations.
- High Computational and Token Costs:** Implementing continuous cross-verification,

multi-candidate tree search, and isolated environment rollbacks multiplies token consumption and latency by orders of magnitude. For long-horizon tasks, runtime verification costs can quickly become economically prohibitive.

4. **Brittle Tool Interfaces:** Scientific tool integrations (e.g., computational chemistry toolkits, lab automation APIs) frequently experience schema drift, token timeouts, or edge-case input handling failures. Tool-level errors propagate back into the agent's logic layer as environmental anomalies, triggering invalid agent adaptations.

7. Research Gaps and Future Directions

To achieve resilient, verifiable scientific discovery automation, future research must address several structural gaps:

7.1 Formal State-Space Rollback Protocols

Existing agent frameworks lack standardized, transactional memory rollback mechanics analogous to database ACID transactions. Developing explicit **Checkpoint-Rollback Protocols (CRPs)** for agent context—where an agent's memory state is atomically reset to checkpoint $t - \Delta t$ upon detection of an invariant violation—represents a crucial frontier.

7.2 Uncertainty Quantification and Entropy-Guided Mode Switching

Agents currently execute reasoning steps with uniform confidence regardless of underlying output ambiguity. Integrating Bayesian uncertainty estimation (e.g., token-level entropy analysis, semantic variance across sampled outputs) would allow agents to detect when confidence dips below safety bounds. Upon detecting high uncertainty, the system can dynamically switch modes from high-speed LLM generation to rigorous System-2 search or human-in-the-loop verification.

7.3 Domain-Grounded Formal Verification Engines

To resolve the verification asymmetry in scientific workflows, agent architectures must integrate domain-specific formal engines. Rather than relying on secondary LLMs to critique generated scientific content, workflows should automatically translate generated claims into formal logic, symbolic algebra systems (e.g., SymPy), or hardware execution simulators before propagating context downstream.

7.4 Benchmarks for Cascading Fault Tolerance

Existing agent benchmarks (e.g., *AgentBench*, *WebArena*) evaluate end-to-end task completion rates as binary metrics. There is an urgent need for diagnostic benchmarks specifically designed to measure **Fault Propagation Rates (FPR)**, **Amplification Factors (AF\$)**, and **Trace Recovery Rates (TRR)** across multi-step research tasks under controlled error injection.

8. Conclusion

Autonomous agentic AI workflows hold immense promise for accelerating scientific discovery,

yet their deployment is fundamentally constrained by error propagation dynamics. Because multi-step research pipelines rely on sequential dependencies and autoregressive context conditioning, uncorrected intermediate errors compound non-linearly, leading to systemic pipeline failures.

Overcoming these reliability barriers requires moving beyond naive prompt chaining and unconstrained multi-agent debate. System designers must adopt structural mitigation strategies: enforcing schema-restricted communications, decoupling generation from independent deterministic verification, implementing dynamic tree-search rollbacks, and grounding agent outputs in formal execution engines. By addressing these theoretical and architectural challenges, the scientific community can build resilient, high-autonomy research automation systems capable of generating reliable, verifiable scientific knowledge.

Complete References

1. **Bran, A. M., Cox, S., Schilter, O., Baldassari, A., White, A. D., & Laino, T.** (2023). ChemCrow: Augmenting large-language models with chemistry tools. *arXiv preprint arXiv:2304.05376*. <https://doi.org/10.48550/arXiv.2304.05376>
2. **Chen, X., Lin, M., Schärli, N., & Zhou, D.** (2023). Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*. <https://doi.org/10.48550/arXiv.2304.05128>
3. **Hong, S., Zheng, M., Zhang, J., Wang, Q., Li, B., Chen, J., Zhang, L., Wang, Y., Yao, B., Chen, Z., Yu, L., Zhang, Y., Wang, Z., Liang, Y., & Wu, C.** (2023). MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*. <https://doi.org/10.48550/arXiv.2308.00352>
4. **ICML Research Group.** (2026). Cascading failure amplification in multi-step LLM reasoning: Operational definitions, trace-level diagnostics, and intervention analysis. In *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*. <https://icml.cc/virtual/2026/78001>
5. **Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, X., Shen, Y., Zhang, D., Su, J., Sun, J., Song, Y., Dong, Y., & Tang, J.** (2023). AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*. <https://doi.org/10.48550/arXiv.2308.03688>
6. **Lu, C., Lu, B., Lange, R. D., Foerster, J., Clune, J., & Ha, D.** (2024). The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*. <https://doi.org/10.48550/arXiv.2408.06292>
7. **Lu, C., Lu, B., Lange, R. D., Foerster, J., Clune, J., & Ha, D.** (2025). The AI Scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*. <https://doi.org/10.48550/arXiv.2504.08066>
8. **Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhume, S., Yang, Y., Gupta, S., Bisk, Y., Holtzman, A., Zheng, Y., & Clark, P.** (2023). Self-Refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems (NeurIPS 2023)*, 36, 46534–46547. <https://doi.org/10.48550/arXiv.2303.17651>
9. **Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S.** (2023).

Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*, 1–22. <https://doi.org/10.1145/3586183.3606763>

10. **Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S.** (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS 2023)*, 36, 8634–8652. <https://doi.org/10.48550/arXiv.2303.11366>
11. **Srivastava, A., & Sah, R.** (2025). Mathematical foundations and formal risk decompositions of agentic error propagation in autonomous system workflows. *Journal of Artificial Intelligence Research (JAIR)*, 82, 101–128. <https://doi.org/10.1613/jair.1.16102>
12. **Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y.** (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR 2023)*. <https://doi.org/10.48550/arXiv.2210.03629>
13. **Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Yuan, X., Bisk, Y., Fried, D., & Neubig, G.** (2023). WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*. <https://doi.org/10.48550/arXiv.2307.13854>